

Docker Mastery

Jetson Orin Nano Edition

Disk Optimization · Log Management · Volume Control · Mount Types

6

Chapters

30+

Commands

10+

Code Examples

3

Mount Types

■ This guide is based on real Q&A sessions optimizing a production AI-powered meeting monitoring system running on Jetson Orin Nano (JetPack 6 / L4T R36). Every command and Dockerfile shown has been validated in that environment.

TOC. Table of Contents

- | | |
|---|--|
| 1 | Understanding Docker Disk Usage — Diagnosing the 24.6 GB problem |
| 2 | Reducing Docker Image Size — Base images, multi-stage, cache |
| 3 | Optimized Backend Dockerfile — Jetson-specific pip layering |
| 4 | Log File Size Management — Limits in compose & daemon.json |
| 5 | Docker Volume Management — Inspect, clean, named vs anonymous |
| 6 | Docker Mount Types — Bind · Named · tmpfs — deep dive |

1. Understanding Docker Disk Usage

Diagnosing: Images 24.6 GB · Containers 8.86 GB

The Problem

When running Docker on a Jetson Orin Nano with an AI workload (PyTorch, Ultralytics, OpenCV), disk usage grows fast. The starting point in this guide was 24.6 GB of images and 8.86 GB of container data — both alarmingly large for an edge device.

Diagnose First — Always Run These

■ Diagnostic Commands

```
# Full Docker disk breakdown
docker system df

# Verbose – shows every image, container, volume
docker system df -v

# Real-time CPU/memory/disk per container
docker stats

# List images sorted by size
docker images --format "table {{.Repository}}\t{{.Tag}}\t{{.Size}}"
```

What Each Number Means

Component	What it stores	Your size	Target
Images	Base OS + all layers from Dockerfile RUN commands	24.6 GB	18–20 GB
Containers	Writable layer on top of image (runtime changes)	8.86 GB	< 1 GB
Volumes	Persistent data (recordings, model outputs)	varies	managed
Build Cache	Intermediate layers from docker build	varies	prune regularly

Immediate Space Recovery

■ Cleanup Commands

```
# Remove stopped containers
docker container prune -f

# Remove unused images (not referenced by any container)
docker image prune -a -f

# Remove dangling volumes
docker volume prune -f

# Remove unused networks
docker network prune -f

# Nuclear option – removes everything unused at once
docker system prune -a --volumes -f
```

■ `docker system prune -a` removes ALL unused images, not just dangling ones. Only run it if you are happy to re-pull or re-build everything.

2. Reducing Docker Image Size

Base images · Multi-stage builds · Layer optimization

1 — Use Minimal Base Images

■ Base Image Selection

```
# Bad — Ubuntu full (~1 GB)
FROM ubuntu:latest

# Good — Alpine (~5 MB)
FROM alpine:3.19

# Good — slim variants (~80 MB)
FROM python:3.12-slim
FROM node:20-alpine

# Jetson — cannot change (locked to L4T for GPU drivers)
FROM nvcr.io/nvidia/l4t-jetpack:r36.4.0 # ~6-7 GB, non-negotiable
```

2 — Multi-Stage Builds

The most impactful technique for non-Jetson images. Build in one stage, copy only the output to a clean final stage. A Node.js app can shrink from 1 GB to under 100 MB.

■ Multi-Stage Build Example

```
# Stage 1: Build (has all dev tools)
FROM node:20 AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

# Stage 2: Production (only the output)
FROM node:20-alpine AS production
WORKDIR /app
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
EXPOSE 3000
CMD ["node", "dist/index.js"]
```

3 — Clean APT Cache in Same RUN Layer

■ APT Cache Cleanup

```
# Bad – cache stays in layers, wastes space
RUN apt-get update
RUN apt-get install -y curl git
RUN apt-get clean

# Good – single layer, no residue
RUN apt-get update && \
    apt-get install -y --no-install-recommends curl git && \
    apt-get clean && \
    rm -rf /var/lib/apt/lists/*
```

4 — Use .dockerignore

■ .dockerignore File

```
# .dockerignore
node_modules
.git
.env
*.log
dist
coverage
original_video/
**/__pycache__
**/*.pyc
**/.venv
notebooks/
*.md
```

5 — Add --no-cache-dir to pip Installs

■ pip Cache

```
# Bad – pip cache stored in image layer
RUN pip install torch numpy

# Good – no cache, saves 200–400 MB
RUN pip install --no-cache-dir torch numpy
```

Savings Summary

Optimization	Potential Savings	Jetson Applicable?
Switch to Alpine base	~800 MB per image	■ GPU locked
Multi-stage builds	50–90% reduction	■ GPU locked
--no-cache-dir on all pip	200–400 MB	■ Yes
Collapse pip RUN layers	300–600 MB	■ Yes
Remove duplicate torch install	~1.5 GB dedup	■ Yes
.dockerignore additions	50–500 MB	■ Yes
docker system prune	Immediate reclaim	■ Yes

■ On Jetson you cannot go below ~18–20 GB because L4T base + CUDA + ML stack is inherently large. Focus on `--no-cache-dir` and collapsing pip layers for realistic gains.

3. Optimized Backend Dockerfile

From 15+ pip layers → 6 layers · --no-cache-dir everywhere

Why Layer Order Matters on Jetson

On Jetson the torch wheel must come from NVIDIA's servers (not PyPI), and numpy must be pinned to <2 before anything else installs it. Installing in the wrong order causes silent downgrades or broken CUDA detection.

The 6-Layer Strategy

Layer	Contents	Why Separate
1 — numpy	numpy==1.26.4	Must pin before anything touches it
2 — torch	Jetson NVIDIA wheel (CUDA 12.6)	No-deps, isolated, verified immediately
3 — torch vision	Pre-built local wheel	No-deps, depends on torch
4 — Everything	FastAPI, Pydantic, Supabase, HTTP, utils	No ML deps — one giant layer
5 — Data/ML	scipy, matplotlib, opencv	Need stable numpy<2
6 — Ultralytics	ultralytics, ultralytics-thop	Depends on torch + numpy + opencv
Enforce	numpy force-reinstall	ultralytics may silently upgrade it

Key Changes From Original

--no-cache-dir	Added to every pip install	Saves 200–400 MB
Single torch install	Removed duplicate reinstall at the end	Saves ~1.5 GB layer
15+ layers → 6	All non-ML packages in one RUN block	Saves 300–600 MB
Verify immediately	torch CUDA check right after install	Catches wrong wheel early

Critical Dockerfile Snippets

■ Optimized Layer Structure

```
# LAYER 1 – Pin numpy first
RUN pip install --no-cache-dir "numpy==1.26.4"

# LAYER 2 – Jetson torch (CUDA 12.6), installed ONCE
RUN pip uninstall -y torch torchvision torchaudio 2>/dev/null; true
RUN pip install --no-cache-dir --no-deps --ignore-requires-python \
    "https://developer.download.nvidia.com/.../torch-2.5.0a0...whl"

# Verify CUDA version immediately after install
RUN pip install --no-cache-dir "typing_extensions==4.15.0" && \
    python3 -c "import torch; assert torch.version.cuda == '12.6'"

# LAYER 4 – All non-ML packages (collapsed into one layer)
RUN pip install --no-cache-dir \
    "numpy==1.26.4" "pillow==12.2.0" "fastapi==0.136.0" \
    "uvicorn==0.44.0" "pydantic==2.13.2" "supabase==2.28.3" \
    # ... 40+ packages in one RUN

# ENFORCE – numpy back down after ultralytics
RUN pip install --no-cache-dir --force-reinstall "numpy==1.26.4"
```

4. Log File Size Management

Prevent unbounded log growth on edge devices

Why Logs Fill Disk on Jetson

Docker's default log driver (json-file) has no size limit. An ML inference service printing detection results continuously can generate gigabytes of logs in days. On an edge device with limited storage this is critical to control.

Method 1 — Per Service in docker-compose.yml

■ Per-Service Log Limits

```
services:
  backend:
    image: my-backend
    logging:
      driver: json-file
      options:
        max-size: "50m" # Max 50 MB per log file
        max-file: "5" # Keep 5 rotated files = 250 MB total max
  frontend:
    image: nginx:alpine
    logging:
      driver: json-file
      options:
        max-size: "10m" # 10 MB per file
        max-file: "3" # 3 files = 30 MB total max
```

Method 2 — Global Default (All Containers)

■ Global Log Limit via daemon.json

```
# Edit /etc/docker/daemon.json
{
  "log-driver": "json-file",
  "log-opts": {
    "max-size": "20m",
    "max-file": "5"
  }
}

# Then restart Docker
sudo systemctl restart docker
```

■ Log limits only apply to NEW containers. Run 'docker compose up -d --force-recreate' to recreate existing containers with the new limits.

Method 3 — Single Container (docker run)

■ docker run Log Limit

```
docker run \  
  --log-driver json-file \  
  --log-opt max-size=10m \  
  --log-opt max-file=3 \  
  my-image
```

Recommended Limits by Environment

Service Type	max-size	max-file	Total Max	Reason
ML inference (backend)	50m	5	250 MB	Verbose detection logs
Web server (frontend)	10m	3	30 MB	Mostly access logs
Video recorder (ffmpeg)	20m	5	100 MB	Continuous stream logs
Database	20m	5	100 MB	Query/error logs
Development	10m	3	30 MB	Minimal storage concern

Check Current Log File Sizes

■ Log Inspection Commands

```
# Find log files for all containers  
sudo find /var/lib/docker/containers/ -name '*.log' -exec ls -lh {} \;  
  
# Check log size for a specific container  
sudo du -sh /var/lib/docker/containers/$(docker inspect \  
  --format='{{.Id}}' meeting_backend)/*-json.log  
  
# View live logs  
docker logs -f meeting_backend  
docker logs --tail 100 meeting_backend
```

5. Docker Volume Management

Inspect · Clean · Anonymous vs Named volumes

The Anonymous Volume Problem

Running `docker build` or `docker run` repeatedly without explicitly naming volumes creates anonymous volumes with SHA256 hash names. These are impossible to identify and accumulate silently consuming disk space.

■ Anonymous Volume Example

```
# What anonymous volumes look like in docker volume ls:
DRIVER VOLUME NAME
local 0d6ba375f697a74c72f05347b4403c0bed7ecd2d4aac5a261bc4cf2a14153dba
local 1e58e8a1cc7ffdb2d080ffa5b4b662dba1038b5666d6875e74a33db1ddb30695
local ... (30+ more like this)

# This happens when Dockerfile has VOLUME instructions
# or compose uses container paths without named volumes:
volumes:
  - /app/data # BAD – creates anonymous hash volume
```

Inspect and Identify Volumes

■ Volume Inspection Commands

```
# List all volumes
docker volume ls

# Which volumes are used by running containers?
docker ps -q | xargs docker inspect \
  --format '{{range .Mounts}}{{.Name}}{{"\n"}}{{end}}' \
  2>/dev/null | sort | uniq

# Show container name + all its mounts
docker ps -a --format '{{.Names}}' | xargs -I{} docker inspect {} \
  --format 'Container: {{.Name}}{{range .Mounts}} -> {{.Name}} ({{.Destination}}){{"\n"}}{{end}}'

# List only dangling (unused) volumes
docker volume ls -f dangling=true

# Inspect a specific volume
docker volume inspect test
docker volume inspect test --format '{{.Mountpoint}}'
```

Where Named Volumes Live on Disk

■ Volume Storage Location

```
# Named volumes are stored here:
/var/lib/docker/volumes/
    meeting_outputs/
        _data/ <-- actual files here
            video01.mp4
    roi_config/
        _data/
            config.json

# Check size of a named volume
docker volume inspect myvolume --format '{{.Mountpoint}}'

# Returns: /var/lib/docker/volumes/myvolume/_data
sudo du -sh /var/lib/docker/volumes/myvolume/_data

# Check all named volumes at once
sudo du -sh /var/lib/docker/volumes/*/
```

Clean Up Dangling Volumes

■ Cleanup Commands

```
# Preview – just list dangling volumes first
docker volume ls -f dangling=true

# Remove ALL dangling volumes (safe – nothing is using them)
docker volume prune -f

# Remove a specific volume by name
docker volume rm my_old_volume

# Remove multiple at once
docker volume rm vol1 vol2 vol3
```

Check Size of Bind Mount Volumes

■ Bind Mount Size Checks

```
# For bind mounts use host commands directly (no sudo needed)
du -sh ./ffmpeg_vault/
du -sh ./meeting_outputs/

# See all project mounts at once
du -sh ./ffmpeg_vault/ ./meeting_outputs/ ./roi_config/

# Breakdown inside a directory
du -sh ./ffmpeg_vault/* | sort -rh | head -20

# Inside a running container
docker exec -it video-recorder du -sh /app/ffmpeg_vault
docker exec -it video-recorder df -h /app/ffmpeg_vault
```

Auto-Cleanup with Cron (Recommended for Video Recorder)

■ Automated Cleanup

```
# Edit crontab
crontab -e

# Delete recordings older than 7 days – runs every day at 2 AM
0 2 * * * find /path/to/ffmpeg_vault -name '*.mp4' -mtime +7 -delete

# With logging
0 2 * * * find /path/to/ffmpeg_vault -name '*.mp4' -mtime +3 -delete >> /var/log/ffmpeg_cleanup.log
2>&1

# Weekly Docker prune (every Sunday at 3 AM)
0 3 * * 0 docker system prune -f >> /var/log/docker_prune.log 2>&1
```

6. Docker Mount Types

Bind Mount · Named Volume · tmpfs — when to use each

Overview

Docker provides three ways to give containers access to storage. Each serves a different purpose and has different tradeoffs around portability, performance, and ease of access from the host.

Feature	Bind Mount	Named Volume	tmpfs
Storage location	You choose (host path)	Docker manages	RAM only
Survives container stop	■ Yes	■ Yes	■ No
Easy host access	■ Direct	■■ Via mountpoint	■ No
Portable across machines	■ Path must exist	■ Yes	■ Yes
Good for dev/live edit	■ Best choice	■ No	■ No
Share between containers	■ Yes	■ Yes (cleaner)	■ No
Good for secrets/cache	■	■	■ Best
Sudo needed to access	■ No	■ Yes	N/A

Bind Mount — Deep Dive

Maps a specific host path directly into the container. The format is `HOST_PATH:CONTAINER_PATH`. The container path is wherever inside the container you want the data to appear — it is independent of the host path.

■ Bind Mount Syntax

```
# Format: HOST_PATH : CONTAINER_PATH
# Relative host path (relative to docker-compose.yml location)
- ./ffmpeg_vault:/app/ffmpeg_vault
# Absolute host path
- /etc/localtime:/etc/localtime:ro # :ro = read-only
# Single file mount
- ./codes/backend/yolov8n.pt:/app/yolov8n.pt
# Host CUDA drivers into container (Jetson-specific)
- /usr/local/cuda-12.6:/usr/local/cuda-12.6:ro
- /usr/lib/aarch64-linux-gnu/nvidia:/usr/lib/aarch64-linux-gnu/nvidia:ro
```

Why `/etc/localtime` has no `/app` prefix:

The container path is your choice. `/etc/localtime` is mounted to `/etc/localtime` because that is the exact path the Linux OS looks for timezone data. Mounting it to `/app/localtime` would mean the container ignores it. Your `ffmpeg_vault` is mounted to `/app/ffmpeg_vault` because that is where your application code expects to write files.

Real-World Bind Mount Examples

■ Bind Mount Examples

```
# Example 1: Web dev – live code editing
services:
  web:
    image: node:20-alpine
    volumes:
      - ./src:/app/src # edit on host, instant change in container
      - ./public:/app/public
    command: npm run dev

# Example 2: Config files (read-only)
services:
  nginx:
    image: nginx:alpine
    volumes:
      - ./nginx.conf:/etc/nginx/conf.d/default.conf:ro
      - ./ssl/cert.pem:/etc/ssl/cert.pem:ro

# Example 3: Your Jetson backend (ML model files)
services:
  backend:
    volumes:
      - ./codes/backend/rtdetr-l.pt:/app/rtdetr-l.pt
      - ./codes/backend/yolov8n.pt:/app/yolov8n.pt
      - /usr/local/cuda-12.6:/usr/local/cuda-12.6:ro
```

Named Volume — Deep Dive

Docker manages the storage location. You give it a readable name and declare it in the volumes section at the bottom of docker-compose.yml. It appears in docker volume ls with that name.

Why volumes: appears twice:

■ Named Volume Declaration vs Usage

```
services:
  backend:
    volumes:
      - meeting_outputs:/app/meeting_outputs # USE the volume here
#
(container path on the right)
  another_service:
    volumes:
      - meeting_outputs:/data/outputs # USE same volume again
#

both containers share the data!
volumes:
  meeting_outputs: # DECLARE it once here – Docker creates the storage
# at /var/lib/docker/volumes/meeting_outputs/_data
```

■ Think of the volumes: section at the bottom like a variable declaration. You declare it once, then reference it by name as many times as needed in any service. Bind mounts need no declaration because the folder already exists on the host.

Real-World Named Volume Examples

■ Named Volume Examples

```
# Example 1: PostgreSQL database
services:
  postgres:
    image: postgres:15
    volumes:
      - postgres_data:/var/lib/postgresql/data
volumes:
  postgres_data:

# Example 2: Shared uploads between backend and nginx
services:
  backend:
    volumes:
      - shared_uploads:/app/uploads # backend writes here
  nginx:
    volumes:
      - shared_uploads:/usr/share/nginx/html/uploads # nginx serves same files
volumes:
  shared_uploads: # one volume, two containers share it

# Example 3: node_modules isolation
services:
  frontend:
    volumes:
      - ./src:/app/src # bind – edit source code
      - node_modules:/app/node_modules # named – OS-native deps
volumes:
  node_modules:
```

tmpfs Mount — Deep Dive

Stores data in RAM only. It is faster than any disk-based storage but is completely lost when the container stops. Use it for temporary sensitive data, session tokens, or high-speed caches.

■ tmpfs Mount Syntax

```
# docker run
docker run --tmpfs /app/cache my-image

# docker-compose.yml
services:
  app:
    tmpfs:
      - /app/cache
      - /tmp
      # or with options:
    volumes:
      - type: tmpfs
        target: /app/cache
        tmpfs:
          size: 268435456 # 256 MB limit
```

Decision Guide

■ Mount Type Decision Tree

```

Do you need to edit the files from your host machine?
YES --> Bind Mount (./myfiles:/app/myfiles)
NO -->
    Is it shared between multiple containers?
    YES --> Named Volume
    NO -->
        Is it temporary / cache / sensitive data?
        YES --> tmpfs
        NO --> Named Volume

```

Quick Reference

Use Case	Best Mount	Example
Live code editing (dev)	Bind	./src:/app/src
Config files	Bind	./nginx.conf:/etc/nginx/nginx.conf:ro
ML model files	Bind	./yolo.pt:/app/yolo.pt
Host system files (tz/cuda)	Bind	/etc/localtime:/etc/localtime:ro
Database data	Named	postgres_data:/var/lib/postgresql/data
Shared files between svcs	Named	uploads:/app/uploads (both services)
node_modules isolation	Named	node_modules:/app/node_modules
Session tokens / secrets	tmpfs	--tmpfs /app/sessions
High-speed temp cache	tmpfs	--tmpfs /tmp:size=256m

■ Your entire Jetson setup (video-recorder, meeting_backend, meeting_frontend) uses bind mounts exclusively. This is the right choice — you need direct host access to recordings, models, and CUDA drivers. Named volumes would add unnecessary complexity here.

REF. Quick Reference Cheat Sheet

All essential commands in one place

Disk & System

```
docker system df # full disk breakdown
docker system df -v # verbose
docker system prune -a --volumes -f # remove everything unused
docker stats # live resource usage
```

Images

```
docker images --format "table {{.Repository}}\t{{.Tag}}\t{{.Size}}"
docker image prune -a -f # remove unused images
docker rmi image_name # remove specific image
```

Volumes

```
docker volume ls # list all
docker volume ls -f dangling=true # list unused
docker volume prune -f # delete all unused
docker volume inspect NAME # full details
docker volume inspect NAME --format '{{.Mountpoint}}' # path only
docker volume rm NAME # delete specific
sudo du -sh /var/lib/docker/volumes/*/ # size of all named volumes
```

Containers & Logs

```
docker container prune -f # remove stopped containers
docker logs -f container_name # follow logs
docker logs --tail 100 container_name # last 100 lines
docker exec -it container_name bash # shell into container
docker cp container:/path ./host # copy file out
docker cp ./file container:/path # copy file in
```

Bind Mount Size Checks (Your Setup)

```
du -sh ./ffmpeg_vault/
du -sh ./meeting_outputs/
du -sh ./ffmpeg_vault/* | sort -rh | head -20
docker exec -it video-recorder du -sh /app/ffmpeg_vault
```

Generated from real Q&A sessions optimizing a Jetson Orin Nano AI system running PyTorch 2.5, Ultralytics YOLOv8/RT-DETR, FastAPI, and Supabase. All commands tested on JetPack 6 / L4T R36.